ARCHITECTURE DATA & IA · BLOC 3 · OCTOBRE 2026

Concevoir et mettre en œuvre des pipelines de données pour l'IA



NappeCast

Assister la recharge des nappes phréatiques par la prévision

Pipeline ETL · ML + Prophet · Production AWS

Repository GIT · https://github.com/RonanGuillou-hub/NappeCast-forecasting

Ronan GUILLOU · Certification RNCP41993

Prévoir le niveau des nappes pour anticiper les restrictions

97 % des départements français sous restriction d'eau

Le besoin
On mesure le niveau des nappes, mais on ne le prévoit pas : les gestionnaires manquent d'anticipation.

La solution
Une prévision à 14 et 30 jours (modèle Prophet), calculée chaque jour à partir de données publiques : Hub'eau et Open-Meteo.

PÉRIMÈTRE DE CETTE PRÉSENTATION

Bloc 3 — Pipelines de données

- Collecte et flux de données (batch)
- Nettoyage et features
- Orchestration avec DAG Airflow
- Qualité, sécurité, conformité
- Supervision, coûts, traçabilité

Traité dans le Bloc 4

- Entraînement et choix du modèle
- Tests unitaires
- Déploiement continu (CI/CD) avec GitActions
- Monitoring des modèles avec Evidently

Un flux batch quotidien, de la source à la prévision

ÉTAPE 1

Sources

- Hub'eau : niveaux des nappes
- Open-Meteo : météo
- API publiques et gratuities
- Relevés journaliers

Externe

ÉTAPE 2

Collecte

- Appel incrémental : seulement les nouveaux jours
- Endpoint FastAPI

S3 · raw/

ÉTAPE 3

Nettoyage

- Colonnes trop vides ou trop corrélées retirées
- Doublons supprimés
- Trous interpolés

S3 · interim/

ÉTAPE 4

Features

- Cumuls de pluie sur 30 et 90 jours
- Features métiers
- Alignement temporel
- Feature memory

S3 · processed/

ÉTAPE 5

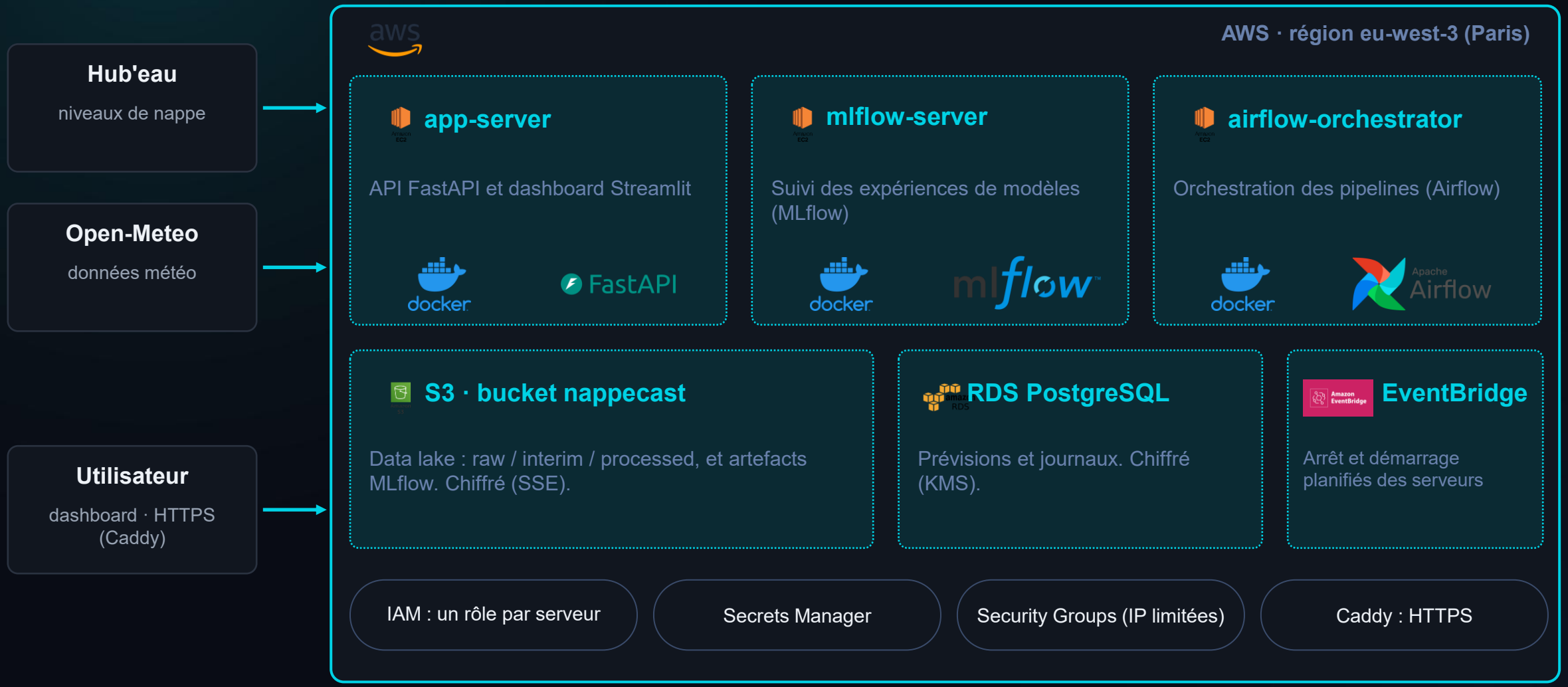
Prévision

- Prévisions à 14 et 30 jours avec Prophet
- Tests avec XGBoost en baseline
- Chargement dans RDS PostgreSQL
- Dashboard Streamlit

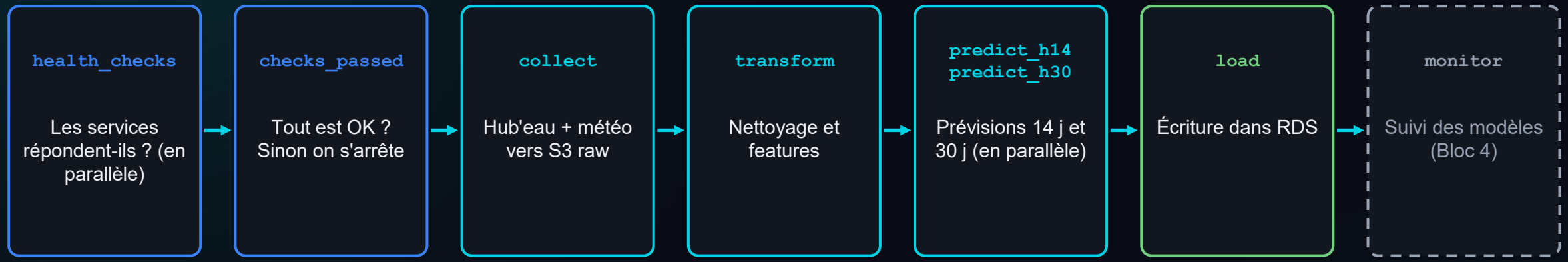
RDS · tables

Batch quotidien · les données brutes sont conservées avant toute transformation (logique ELT) · clés S3 horodatées : préfixe/AAAA/MM/nom_AAAAMMJJ

Trois serveurs, un data lake, une base de données



Le DAG « Daily Predict » : tous les jours à 06:00



Airflow orchestre, l'API exécute

Chaque tâche appelle un endpoint de l'API (/pipeline/collect, /transform, /forecast, /load). Un en-tête secret protège ces appels : sans lui, l'API répond 403.

Planification et robustesse

- Exécution quotidienne à 06:00
- Une seule exécution à la fois
- Délais maximum (timeouts) sur chaque tâche
- Arrêt si un contrôle de santé échoue

Second DAG : les journaux

Toutes les heures, les journaux de l'API sont transférés vers S3 (endpoint /data/transfert_log). On garde ainsi une trace de chaque exécution, même si le serveur est arrêté.

Des données fiables, un accès maîtrisé

Qualité des données

- Contrôles de santé avant chaque exécution
- Environ 62 tests unitaires sur le pipeline : valeurs manquantes, doublons, fréquence journalière

Sécurité

- Un rôle IAM par serveur
- Secrets hors du code (Secrets Manager)
- En-tête secret sur les appels de pipeline
- Accès réseau limités par IP (variables)
- RDS chiffré (KMS), S3 chiffré (SSE)

Conformité RGPD

- Données publiques et libres : Hub'eau, Open-Meteo
- Aucune donnée personnelle collectée
- Infrastructure décrite en code (Terraform), sans secret dans le dépôt

AXES D'AMÉLIORATION IDENTIFIÉS

Ajouter une validation de schéma et une détection d'anomalies sur les données · restreindre l'accès public à la base RDS · réduire les droits du rôle d'administration EC2

Mesurer, maîtriser les coûts, pouvoir tout retracer

Supervision

- Journaux de l'API transférés toutes les heures vers S3
- Historique de chaque exécution dans Airflow
- RDS : Performance Insights et métriques toutes les 60 s

Coûts

- Arrêt et démarrage planifiés des serveurs (EventBridge)
- Services managés dimensionnés petits
- Budget mensuel : 30€

Traçabilité (data lineage)

- Couches S3 raw / interim / processed, horodatées
- Chaque donnée remonte à sa source et à sa date
- Infrastructure versionnée : Terraform + Git

Principe · chaque exécution laisse une trace : journaux horaires dans S3, historique des tâches dans Airflow, données horodatées par couche. On peut rejouer et expliquer n'importe quelle prévision.

Un projet piloté en 4 sprints dans Notion

SPRINT 1

Filet de sécurité

Livré · 23 juillet

- Backfill de l'historique des piézomètres
- Dépôt des données brutes en S3
- Ressources S3 et IAM de base
- Alarme budgétaire
- EventBridge : extinction des EC2 la nuit
- Streamlit light

SPRINT 2

Version ambitieuse

Livré · 29 juillet

- Migration de MLflow vers AWS
- Architecture de production cible
- Endpoint d'inference
- Amélioration Streamlit

SPRINT 3

Lead

Livré · 20 septembre

- Serveur et orchestration Airflow
- Serveur RDS et table de prévisions
- Endpoints de l'API
- Logs vers S3 via Airflow
- + entraînement et CI/CD (Bloc 4)

SPRINT 4

Buffer

Backlog : ce qu'on aurait pu ajouter

- Terraform (IaaS)
- Tests anti-fuite temporelle
- Qualité des données en SQL
- Ajustement des permissions IAM
- Choix du piezomètre

Méthode · user stories « En tant que… je veux… », epics (Collecte, Feature engineering, Infrastructure, Visualisation), points d'effort, trois contributeurs : Ronan, Laura, Adrien

Démonstration vidéo (moins d'une minute)



Vidéo à insérer

Ce que montre la vidéo

1. Console AWS : les 3 EC2 et RDS en service
2. Airflow : le DAG Daily Predict réussi
3. S3 : les fichiers du jour dans raw / interim / processed
4. Dashboard Streamlit : la prévision à 14 et 30 jours